

Presentation 1

You are once again programming the data-acquisition system on a probe that will be performing a close flyby of the planet Mars. This probe also flies over the Martian surface at a rate of 1 km/s, passing over n 1-km sectors before it leaves the atmosphere. It features an improved radar system that is able to perform scans more rapidly than the previous probe, with less time spent recharging.

This radar system is capable of two scanning modes, a surface scan and a deep geological scan. The surface scan can be used once every second without any charging or recharging time. The deep geological scan requires that the system charge a radar pulse for one second *before* it can be used, during which time it is inoperative (so, if a deep geological scan is performed on Sector i , Sector $i - 1$ cannot be scanned at all). Once again, the planetary science team has created point value estimates for the information they can gain by performing either a deep geologic scan, or a surface scan, of each sector.

The point values are stored in two arrays, $S = [s_1, s_2, \dots, s_n]$ for shallow scans, and $D = [d_1, d_2, \dots, d_n]$ for deep scans. For each sector i , you must choose whether to perform a shallow scan (learning the information value s_i), perform a deep scan (learning the information value d_i), or charge (learning 0 information), with the restriction that if a deep scan is performed on Sector i then a charging action must've been performed on Sector $i - 1$.

Design an efficient algorithm to create a schedule (sequence of actions CHARGE, SURFACE, or DEEP) that maximizes the total information gain for the flyby.

Example:

Input: $n = 3$, $S = [2, 3, 4]$, $D = [5, 10, 7]$.

Output:

- Charge while passing over Sector 0...
- ... in order to perform a deep scan of Sector 1 ($d_1 = 10$).
- Then, perform a shallow scan of Sector 2 ($s_2 = 4$).

The total information gain would be $0 + 10 + 4 = 14$.

Presentation 1 Solution

1: The "last" decision

The last decision involves determining whether to perform a surface scan, a deep scan, or no action at all in the final sector n to maximize information gained.

2: Define subproblems

$g[i]$: the maximum information that can be obtained up to sector i .

$a[i]$: the action (surface scan, geological scan, or charging) made over Sector i , that leads to the maximum information obtained up to sector i .

3: Develop recursive formulation to solve subproblems

$$p[i] = \begin{cases} 0 & \text{if } i < 0 \\ s_i & \text{if } i = 0 \\ \max(s_i + p[i - 1], d_i + p[i - 2], p[i - 1]) & \text{otherwise} \end{cases} \quad (1)$$

$$a[i] = \begin{cases} \text{CHARGE} & \text{if } p[i] = p[i - 1] \\ \text{SURFACE} & \text{if } p[i] = s_i + p[i - 1] \\ \text{DEEP} & \text{if } p[i] = d_i + p[i - 2] \end{cases} \quad (2)$$

4: Pseudocode

We can implement this formulation in a bottom-up manner as follows:

Algorithm 1 Compute Optimal Scan Process

```

1: function COMPUTESCAPS( $S, D, n$ )
2:    $p \leftarrow$  Array of length  $n$ 
3:    $p[0] \leftarrow s[0]$ 
4:   for  $i \leftarrow 1$  to  $n$  do                                      $\triangleright$  Compute value using recurrence relation
5:     if  $i - 2 < 0$  then
6:        $p[i] \leftarrow \max(S[i] + p[i - 1], D[i], p[i - 1])$ 
7:     else
8:        $p[i] \leftarrow \max(S[i] + p[i - 1], D[i] + p[i - 2], p[i - 1])$ 
9:     end if
10:  end for
11:   $a \leftarrow$  Array of length  $n$ 
12:  for  $i \leftarrow 1$  to  $n$  do                                      $\triangleright$  Reconstruct the optimal scanning procedure
13:    if  $i > 1$  then
14:      if  $p[i] = D[i] + p[i - 2]$  then
15:         $a[i] \leftarrow \text{DEEP}$ 
16:      end if
17:    end if
18:    if  $p[i] = S[i] + p[i - 1]$  then
19:       $a[i] \leftarrow \text{SURFACE}$ 
20:    end if
21:    if  $p[i] = p[i - 1]$  then
22:       $a[i] \leftarrow \text{CHARGE}$ 
23:    end if
24:  end for
25:  return  $a, p[n]$ 
26: end function

```

5: Show execution on an example

Consider the example given in the problem statement: $n = 3$, $S = [2, 3, 4]$, $D = [5, 10, 7]$.

The following tables should be computed progressively:

i	-1	0	1	2
s_i	—	2	3	4
d_i	—	5	10	7
$p[i]$	0	5	10	14
$c[i]$	—	DEEP	DEEP	SURFACE

(3)

The optimal project schedule can be reconstructed as follows: CHARGE, DEEP, STANDARD, yielding a total information gain of 14 points.

6: Proof

6A: Inductive proof of correctness

Base case: When $i < 0$, $p[i]$ is 0, since no scans can be performed yet. When $i = 0$, we can either choose a surface scan (s_0), a geologic scan (d_0), or to charge the scanner (0 information). Thus, $p[0]$ correctly represents the maximum information obtainable from the first sector, and $a[0]$ correctly indicates the choice that leads to this result.

(Strong) Inductive Hypothesis: Assume $p[0 \dots i]$ and $a[0 \dots i]$ are correct for some arbitrary i . What about $i + 1$?

Inductive Step: For sectors $i + 1$, we consider three scenarios: taking a surface scan (s_{i+1}), taking a geologic scan (d_{i+1}), which requires charging while flying over Sector i , or charging the scanner. The information from Sector $i + 1$ when choosing a surface scan, includes the information up to Sector i plus s_{i+1} , and for a deep scan, it includes the information up to Sector $i - 1$ plus d_{i+1} , with the time over Sector i having been spent charging. The decision to charge the scanner considers the maximum information up to Sector i , without adding any information for the current sector ($i + 1$).

This scenario can be expressed as:

$$p[i + 1] = \max(s_{i+1} + p[i], d_{i+1} + p[i - 1], p[i])$$

Given the strong inductive hypothesis, we know $p[i]$ and $p[i - 1]$ are correctly computed. Thus, evaluating the maximum of these three scenarios will yield the correct value for $p[i + 1]$, ensuring that the choice $a[i + 1]$ aligns with this maximum information decision.

6B: Proof of Optimal Substructure

The subproblems in this problem are independent because our options when flying over each sector, are determined only by our actions in *previous* sectors (namely, whether we have charged the scanner or not), and not sectors we will pass over in the future.

To prove optimal substructure, we will first assume that R is an optimal solution for the problem covering Sectors $1 \dots n$. Then, we will define R' as the actions of R as applied to the subproblem of Sectors $1 \dots n - 1$. We will then posit the existence of a solution R'' applied to Sectors $1 \dots n - 1$, which recovers more information than R' . Depending on the charging status of the sensor in Sector $n - 1$, one of 3 cases can occur:

1. If R' did not end in CHARGE, then R did not originally end with a deep scan. We can construct a new solution R^* , based on taking one additional action after R'' , either a deep or shallow scan, whichever returns the greater amount of information. Since we could *only* take a shallow scan as the last action in R , the information value of R^* must be greater than R 's. However, R was assumed to be the optimal solution that contained the maximum possible value of information over Sectors $1 \dots n$ - a contradiction. Therefore, no R' can exist.
2. If R' and R'' both end in CHARGE, the options available for Sector n in R^* are the same as those in R . Since R'' produces more information over Sectors $1 \dots n - 1$, R^* must have a higher information yield overall. Once again, since R was assumed to be the optimal

solution that contained the maximum possible value of information over Sectors $1...n$, this is a contradiction, and no better subproblem solution R' can exist.

3. If R' ends in CHARGE, and R'' does not, then R^* will not be able to end in a deep scan even if R did, so the information value of R^* could be arbitrarily lower than R . However, in this case R'' is not an *applicable* solution to the subproblem over Sectors $1...n - 1$, because the action performed over Sector n must be different than it was in R . So, we do not need to consider this case.

7: Runtime Analysis

The dynamic programming table computation involves:

- Iterating through each of the n sectors to determine $p[i]$, based on the information and choices of previous sectors. Each sector is evaluated in constant time (comparing a maximum of 3 previous sectors' values).
- Iterating through each of the n sectors to fill in $c[i]$. Once again, each sector is evaluated in constant time (comparing a maximum of 3 previous sectors' values).

So, the total runtime of the algorithm, including both computing the dynamic programming table and reconstructing the optimal schedule, is $O(n) + O(n) = O(n)$.

Presentation 2

You are tasked with coordinating the programming for a local television station covering the 2028 Olympic Games. n events are running throughout the day, some overlapping, and you have only one camera crew available. You have a ratings survey from a previous year's Olympics that provides good estimates of how many viewers will tune in for each event, and a schedule for the events in 1-hour increments. Assume that an event needs to be filmed start-to-finish if you film it at all, and the time taken to travel to another event is negligible. You can choose to spend time not filming anything at all, in which case the station will run other programming, but views for that programming will not be credited to the sports desk. You will need to determine a filming schedule for the camera crew that will maximize the expected number of viewers.

We will represent the schedule using three arrays S, E , and V , each of size n , such that Event i begins at hour $S[i]$, ends at hour $E[i]$, and will bring in a *total* viewership over its entire runtime of $V[i]$. Two events i and j are considered overlapping if they share an hour, i.e. if $\exists m$ such that $S[i] \leq m < E[i], S[j] \leq m < E[j]$.

You can assume that the schedules are sorted in any way that is convenient for you, e.g., your algorithm can take the input sorted by start time (so S is in non-decreasing order) or end time (so E is in non-decreasing order), or any other well-defined ordering.

Design an efficient algorithm to calculate the optimal filming schedule.

Example:

Input:

- $n = 4$
- $S = [1, 3, 4, 8]$
- $E = [4, 7, 12, 9]$
- $V = [50, 110, 200, 30]$
- That is, Event 1 runs from 1:00 to 4:00 and will bring in about 50 viewers; Event 2 runs from 3:00 to 7:00 and will bring in 110 viewers, etc.

Output: Filming Events 1 and 3 is optimal with $V[1] + V[3] = 250$ total views. Filming these two offers is feasible as the intervals $[1, 4)$ and $[4, 12)$ do not overlap. However, for example, we cannot film both Event 2 and Event 3, as the intervals $[3, 7)$ and $[4, 12)$ overlap (they are both occurring simultaneously over hours 4, 5 and 6).

Presentation 2 Solution

This solution will assume that the events are sorted by *ending* time (E is in non-decreasing order)

1: The “last” decision

The last decision is to choose the final event to film, ensuring it does not overlap with previously filmed events, to maximize the total viewership.

2: Define subproblems

Let $dp[i]$ represent the maximum viewership possible by considering events up to the i th event, and then filming the i th event.

Define $choice[i]$ as the index of the last even filmed *before* the i th event, to obtain viewership $dp[i]$.

3: Develop recursive formulation to solve subproblems

$$dp[i] = \begin{cases} 0 & \text{if } i = 0 \\ V[i] + \max_{j < i, E[j] \leq S[i]} (dp[j]) & \text{otherwise} \end{cases} \quad (4)$$

$$choice[i] = \begin{cases} \text{NIL} & \text{if } i = 0 \\ \arg \max_{j < i, E[j] \leq S[i]} (dp[j]) & \text{otherwise} \end{cases} \quad (5)$$

4: Pseudocode

We can implement this formulation in a bottom-up manner as follows:

Algorithm 2 Maximize Viewership

```

1: function MAXIMIZE_REVENUE( $S, E, V, n$ )
2:    $dp \leftarrow$  Array of length  $n$ 
3:    $choice \leftarrow$  Array of length  $n$ 
4:    $dp[0] \leftarrow 0$ 
5:    $choice[0] \leftarrow \text{NIL}$ 
6:   for  $i \leftarrow 1$  to  $n$  do                                      $\triangleright$  Compute value using recurrence relation
7:      $bestViews \leftarrow 0$ 
8:      $bestEvent \leftarrow \text{NIL}$ 
9:     for  $j \leftarrow 1$  to  $i - 1$  do
10:      if  $E[j] \leq S[i]$  and  $dp[j] > bestViews$  then
11:         $bestViews \leftarrow dp[j]$ 
12:         $bestEvent \leftarrow j$ 
13:      end if
14:       $dp[i] \leftarrow bestViews + V[i]$ 
15:    end for
16:  end for
17:   $F \leftarrow \{\}$ 
18:   $i \leftarrow \arg \max_{j < n} \{dp[j]\}$                                 $\triangleright$  Reconstruct the optimal solution
19:  while  $i > 0$  do
20:    Append  $i$  to the front of  $F$ 
21:     $i \leftarrow choice[i]$ 
22:  end while
23:  return  $S$ 
24: end function

```

5: Show execution on an example

Consider the example given in the problem statement: $n = 4$, $S = [1, 3, 4, 8]$, $E = [4, 7, 12, 9]$, $V = [50, 110, 200, 30]$.

The following tables should be computed progressively:

i	0	1	2	3	4
$S[i]$	—	1	3	8	4
$E[i]$	—	4	7	9	12
$V[i]$	—	50	110	30	200
$dp[i]$	0	50	110	140	250
$choice$	<i>NIL</i>	0	0	2	1

(6)

Since the maximum value of dp is 250 at index 3, we start there.

- We begin by adding 3 to the solution.
- $choice[3] = 1$, so we now go to index 1.
- We add 1 to the solution.
- $choice[1] = 0$, and 0 is our base case, so we return the solution.

6: Prove correctness

6A: Inductive proof of correctness

We want to show that our recurrence correctly selects the events that maximize the total viewership for each i without overlap.

Base case: $i = 0$, this represents no events being filmed, and the viewership is correctly 0. $dp[i]$ is accurately 0, as there are no events to film, which aligns with the initial condition.

(Strong) Inductive Hypothesis: Assume $dp[0 \dots i]$ is correct for some arbitrary i . How about $i + 1$?

Inductive Step: When considering event $i + 1$, we aim to maximize the total viewership while ensuring no overlap with events we are already filming, and assuming that we film event $i + 1$. So, we want to maximize the previous choices such that the last chosen event does not overlap with event $i + 1$. This leads to the expression:

$$dp[i + 1] = V[i + 1] + \max_{j < i+1, E[j] \leq S[i]} (dp[j])$$

Since this is the formula we apply to the DP table, by the strong inductive hypothesis, we have correctly computed $dp[j]$ for all $j < i + 1$. Consequently, $dp[i + 1]$ will be the maximum viewership obtainable up to the $(i + 1)^{\text{th}}$ event, including the $(i + 1)^{\text{th}}$ event.

6B: Proof of Optimal Substructure

We will define S as the optimal solution to the problem over events $1 \dots n$, with the restriction that Event n is filmed. S' is then the optimal solution to the problem over events $1 \dots n - 1$, which preserves the property that the last event in S' cannot intersect with event n . Then, we could construct a new solution S^* , to the problem over events $1 \dots n$, which films all of the events in S' and also films Event n . If S'' brings in more viewers than S' and both film Event n (adding $V[n]$ to that total), then S^* has more viewers overall than S . However, we had gone into this problem with the assumption that S was optimal. So, since assuming S'' exists results in a contradiction of that assumption, no S'' can exist.

7: Runtime Analysis

To compute the DP table for the events:

- We iterate through each event i to determine $dp[i]$.
- For each event i , we potentially need to check all previous event to find the maximum $dp[j]$ for $j < i$ where the events do not overlap.

Thus, for each i , the worst-case scenario involves iterating through $i - 1$ previous events.

Therefore, computing the DP table will take at most:

$$T(n) = \sum_{i=1}^n \Theta(i) = \Theta(n^2) \quad (7)$$

time.

Since reconstructing the optimal solution involves tracing back through the choices made (stored in an array or through some other means), and we may have to trace back through all n events in the worst case, which takes $\Theta(n)$ time.

Thus, the total worst-case runtime of the algorithm is $\Theta(n^2) + \Theta(n) = \Theta(n^2)$.